

352 Project2

CHRISTINE YUE, ZAHIR MUSLIWALA

March 2025

1 Team details

Christine Yue 207000676, Zahir Musliwala 209007164

2 Collaboration

Christine and Zahir collaborated on this project together. We looked at the Socket Programming Lecture Slides for socket implementation, Project 2 slides provided by Negin Dehghanchaleshtori for detailed information and instructions on how to best execute the project, GeeksforGeeks for general error debugging (such as ConnectionRefusedError), and YouTube for comprehending TCP servers and clients.

3 Query Functionality

We implemented recursive and iterative query functionalities based on the flag parameter (**rd** for recursive and **it** for iterative) specified in the query.

Recursive Queries (rd):

1. Client sends the query to the root server (RS) using the **send_query** function.
2. The RS processes the query in the **handle_query** function:
 - ***If the domain belongs to a top-level domain server (TS)***, the RS forwards the query to the appropriate TS using the **send_query_to_ts** function.
 - The TS checks its database and returns **aa** (authoritative answer) if the domain is found or **nx** if it does not.
 - If the TS responsive is authoritative, the RS modifies the flag to **ra** (recursive answer) and sends the resolved IP back to the client.
 - ***If the domain does not belong to a TS***, the RS resolves the query using its own database (rsdatabase.txt) and returns **aa** if the domain is found or **nx** if it does not.
3. Therefore, the RS handles the entire resolution process, including querying to the TS if necessary.

Iterative Queries (it):

1. Client sends the query to the RS using the **send_query** function.
2. The RS processes the query in the **handle_query** function:
 - If the domain belongs to a TS, the RS returns a referral to the appropriate TS with an **ns** flag.
3. The client increments the identification and directly queries the TS using the **send_query** function.
4. The TS checks its database and returns **aa** if the domain is found or **nx** if it does not.
5. Therefore, the client handles the final resolution step by querying the TS directly after receiving the referral from the RS.

4 Any Issues

We have developed and executed custom test cases to ensure comprehensive validation of our code across multiple machines. These tests verify that the machines align with the configurations in `rsdatabase.txt` and that the client communicates correctly with the root server (RS). This thorough testing process confirms that our implementation meets the specified requirements and functions as intended.

Our testcases: `hostnameesting.txt` and `testcases.txt`

These files are not included in the submission due to submission guidelines. If access to these test cases is required, please feel free to reach out to us.

5 Difficulties

In order to make our code align with the example inputs and outputs given, we executed our program as follows:

```
(on java.cs.rutgers.edu) python3 ts1.py 45000
(on cheese.cs.rutgers.edu) python3 ts2.py 45000
(on ilab4.cs.rutgers.edu) python3 rs.py 45000
(on ilab1.cs.rutgers.edu) python3 client.py ilab4.cs.rutgers.edu 45000.
```

The provided instructions (different from the above) caused a conflict between the root server and `ts2`, because they were running on the same hostname. This issue highlighted the necessity of running the `rs`, `ts1`, and `ts2` on different `ilab` machines. Furthermore, the `ts1` and `ts2` must run on the machines defined in `rsdatabase.txt` to ensure proper functionality and avoid conflicts. This experience reinforced the importance of correctly configuring and distributing the servers across distinct machines.

6 Learn

Through this project, we learned to carefully analyze test cases, as they may not always align with the instructions, and the importance of writing flexible,

non-hardcoded code to ensure compatibility across different sets of iLab machines and database configurations. We gained hands-on experience managing distributed systems across multiple iLab machines, utilizing SCP for efficient file transfers between local and remote servers, and editing files directly on remote systems using Vim. These skills have significantly enhanced our ability to work effectively in distributed computing environments.